

‘O que é lógica de programação?

Pensar e construir uma solução para o problema

- **Algoritmo:** Do jeito que a gente organiza, o passo a passo de forma sequencial para resolver o problema com ajuda do computador.

Ele precisa ser:

- Claro;
- Lógico;
- Executável.

Algoritmo

Todos algoritmo precisa de três coisas:

- **Entrada(INPUT):** Tudo o'que o programa precisa receber do usuário ou de algum lugar para poder funcionar;
- **Processamento:** Onde acontece o trabalho, cálculo, transformações da informação em si (o cérebro do algoritmo). O processamento é feito por meio de operações, decisões, cálculos ou repetições.
- **Saída(OUTPUT):** Onde o programa entrega de volta ao usuário: respostas, resultado, mensagens

Como declarar uma variável e atribuir valor nela

- **Variável:** é como uma caixinha com nome, onde você guarda uma determinada informação;

idade < 12 = forma portugues estruturado, é assim que a variável recebe o seu valor

Como escrever um decisão **(SE & SENÃO)**:

```
Início
Escreva("Está chovendo?")
Leia(resposta)
Se resposta = "sim" então
    Escreva("Leve o guarda-chuva")
Senão
    Escreva("Pode sair de boa pq não vai chover")
Fim
```

Como escrever uma estrutura de repetição(Loop) **(ENQUANTO)**:

```
Início
Escreva("Está sol? (sim/não)")
Leia(tempo)

Enquanto tempo = "sim" faça
    Escreva("Pular na piscina!")
    Escreva("Ainda está sol? (sim/não)")
    Leia(tempo)
Fim
```

Linguagem de alto e baixo nível

- Baixo nível: Ligação direta com o hardware (alta performance + complexa)

- Alto nível: Tem uma camada, para fazer a conversão do alto nível para o baixo nível

IDE e compilador

- IDE: Ambiente onde iremos digitar o nosso código;
- Compilador: Converter o código em binários (0 & 1)

Temos 3 formas de usar a IDE no nosso curso em C:

- **IDE online:** Pesquisa no google um compilador em C;
- **Dev C++:** Pesquisa no google dev c++, entra na página (SourceForge - dev C++ download);
- **Vs code:** Têm seguir umas cláusulas para instalar o compilador, também tem que procurar no google.

Hello World

```

1 #include <stdio.h> //include: incluindo no programa - <stdio.h> = biblioteca
2
3 int main() // int: tipo de dado, o tipo de dado que irá retornar, main: declaração de função principal
4 {
5     printf("Hello World!"); // metodo de saída -- que é exibido na tela
6     return 0; // sempre se que retorna
7 }

```

Estrutura Básicas de Controle

- Tipos de Dados em C

LEMBRA-SE: caso tenha que passar o tamanho da sua string (por exemplo: char nome [8]), sempre tem que passar um valor a mais, pois na linguagem C, sempre terminar com o barra zero;

Tipo	Tamanho (bytes)	Faixa de Valores (Aproximada)	Exemplo de Uso
char	1 byte	-128 a 127 (signed char) 0 a 255 (unsigned char)	char letra = 'A';
int	4 bytes	-2.147.483.648 a 2.147.483.647	int idade = 25;
float	4 bytes	~6 casas decimais de precisão	float pi = 3.1415;
double	8 bytes	~15 casas decimais de precisão	double numero = 2.718281828;
_Bool	1 byte	0 (falso) ou 1 (verdadeiro)	_Bool ligado = 1;

```

2 ESTRUTURA_BASICAS > tipodados > tipodados.c > main()
1  #include <stdio.h>
2
3  int main()
4  {
5      int numero = 10;
6      float numeroQuebrado = 50.49;
7      char letra = 'A';
8      // char nome = "Maria"; -- isso se torna ERRADO, pois as aspas, simplifica uma STRING e o CHAR espera apenas 1 caracter
9      char nome[6] = "Maria";
10     // Como estou usando string, eu passo o tamanho da string dentro de conchete
11     // (LEMBRA-SE: sempre tem que passar um valor a mais, pois na linguagem C, sempre terminar com o barra zero)
12
13     printf("Numero: %d \n", numero); // \n, serve para pular uma linha
14     printf("Flutuante: %.2f \n", numeroQuebrado); // .2, serve para deixar duas casas decimais depois do ponto
15     printf("Letra: %c \n", letra);
16     printf("Nome: %s \n", nome); // Como esta recebendo uma string ponho %s
17
18     return 0;
19 }

```

● Entrada e Saída

```

2 ESTRUTURA_BASICAS > entrada_saida > entrada_saida.c > main()
1  #include <stdio.h>
2
3  int main()
4  {
5      int numero;
6
7      printf("Digite um numero inteiro: ");
8      scanf("%d", &numero);
9      //scanf, colocando um código formatado,
10     //%d o tipo de dados que ele vai receber,
11     //&numero, o valor será armazenado nessa variável, por conta do &, que é uma ancora e armazena na memória
12
13     printf("O numero foi o : %d", numero);
14
15     return 0;
16 }

```

● Operadores lógicos e condicionais

```

if ( SE FOR VERDADEIRO ) {
    - Entra nesta condição
} else {
    - Se for falso, entra aqui
}
return 0; }

```

```

int main()
{
    int idade;

    printf("Digiti a sua idade: ");
    scanf("%d", &idade);

    if (idade >= 18)
    {
        printf("MAIOR");
    }
    else
    {
        printf("MENOR");
    }

    return 0;
}

```

```

= -> atribuição
== -> igualdade
> -> maior
< -> menor
>= -> maior ou igual
<= -> menor ou igual
! -> diferente
&& -> AND/E
|| -> OR/OU

```

● Operadores matemáticos

```
RUTURA_BASICAS > operadores > C matematicos.c > ...
#include <stdio.h>

int main()
{
    int num1 = 10, num2 = 2; // declaração de variáveis do mesmo tipo
    int resultado; // posso criar uma variável para guarda resultado

    resultado = num1 + num2; // fazer o calculo aqui

    printf("Adicao: %d + %d = %d\n", num1, num2, resultado); // e passar aqui o baixo
    // printf("Adicao: %d + %d = %d\n", num1, num2, num1 + num2); ou posso usar desta forma também

    printf("Subtracao: %d - %d = %d\n", num1, num2, num1 - num2);
    printf("Multiplicacao: %d * %d = %d\n", num1, num2, num1 * num2);
    printf("Divisao: %d / %d = %d\n", num1, num2, num1 / num2);

    // MÓDULO, MUITO USADO PARA DESCOBRIR SE O NÚMERO É PAR OU ÍMPAR
    printf("Modulo: %d %% %d = %d\n", num1, num2, num1 % num2); // é o resto que sobra da divisão

    return 0;
}
```

```
// MÓDULO, MUITO USADO PARA DESCOBRIR SE O NÚMERO É PAR OU ÍMPAR
// (retorna o resto da divisão)
```

● Switch..Case

```
int main (){
    int opcao = 1;

    //estrutura de controle
    switch (opcao)
    {
        // aqui ele percorre, até chegar o numero que vou declarado
        case 1 : // caso o valor opção for 1, ele entra nessa condição
            printf("Fase numero 1 ");
            break; // usado para finalizar e não ficar percorrendo eternamente, caso o numero ja for encontrado.
        case 2 : // caso o valor opção for 2, ele entra nessa condição
            printf("Fase numero 2 ");
            break; // usado para finalizar e não ficar percorrendo eternamente, caso o numero ja for encontrado.
        case 3 : // caso o valor opção for 3, ele entra nessa condição
            printf("Fase numero 3 ");
            break; // usado para finalizar e não ficar percorrendo eternamente, caso o numero ja for encontrado.
        case 4 : // caso o valor opção for 4, ele entra nessa condição
            printf("Fase numero 4 ");
            break; // usado para finalizar e não ficar percorrendo eternamente, caso o numero ja for encontrado.
        default: // ele entra aqui, quando nenhuma das opção de cima, forem verdadeiras
            printf("Opção Invalida");
    }
}
```

● Loop For

Diagram illustrating the components of a `for` loop:

```
Variable      Condition  Increment
  ↓           ↓          ↓
for(Integer i = 0; i < 5; i++){
    //Code Block
}
```

- FOR: PARA;
- Passa a variável, dando inicialização de onde e qual valor vai começar;
- A condição que esse loop vai percorrer;
- E a Incrementação, para acontecer o loop.

```
#include <stdio.h>

int main()
{
    for (int i = 1; i <= 10; i++) // posso iniciar com i = 1 dessa forma
    {
        // printf("Aluno: %d\n", i + 1); ou dessa forma , onde no (int i = 0 ) e aqui e, baixo ele seria soma, e retornaria o i = 1 .
        printf("Aluno: %d\n", i);
    }

    return 0;
}
```

● Loop Dinâmico

Deixar que o usuário decida quanto vezes o loop vai rodar:

```

2 ESTRUTURA_BASICAS > loop > C loop_dinamico.c > ...
1  #include <stdio.h>
2
3  int main()
4  {
5      int dias;
6
7      printf("Digite quantos dias de treinamento: ");
8      scanf("%d", &dias);
9
10     for (int i = 0; i < dias; i++)
11     {
12         printf("Dia %d: ta pago !\n", i + 1);
13     }
14     printf("\nTreinamento concluido!");
15
16     return 0;
17 }

```

- **Loop Decremento**

```

1  #include <stdio.h>
2
3  int main()
4  {
5      printf("Contagem regressiva para a bomba explodir: \n");
6
7      for (int index = 10; index >= 1; index--) // inicializa a variavel em 10 e vai decrementando ate acabar.
8      {
9          printf("%d...\n", index);
10     }
11     printf("## BOOM! ##\n");
12     return 0;
13 }

```

- **Loop For Alinhado**

```
2 ESTRUTURA_BASICAS > loop > loop_for_alinhado.c > main()
3
4 int main()
5 { // iteração, esta iterando, executando varias vezes
6     for (int i = 0; i <= 10; i++)
7     {
8         printf("\n=====");
9         printf("\nTabuada do %d\n", i); // aqui ele inicializa qual tabuada vai executar e entra dentro do for de j
10
11         for (int j = 0; j <= 10; j++) // ele vai incrementar o j ate 10
12         {
13             printf("%d x %d = %d\n", i, j, i * j); // (i x j = i*j), vai utilizar o valor de i que foi inicializado,
14             // e vai incrementando o j e multiplicando ate j chegar em valor 10, depois finaliza este loop e retorna
15             // pro loop de i e continua fazendo a mesma coisa, ate que o loop de i chegue a 10
16         }
17     }
18
19     return 0;
20 }
```

PROBLEMAS SAÍDA CONSOLE DE DEPURACÃO TERMINAL PORTAS

Tabuada do 1

```
1 x 0 = 0
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = 5
1 x 6 = 6
1 x 7 = 7
1 x 8 = 8
1 x 9 = 9
1 x 10 = 10
```

=====

Tabuada do 2

```
2 x 0 = 0
2 x 1 = 2
```

```
1 #include <stdio.h>
2
3 int main() {
4     for (int i = 0; i <= 10; i++) {
5         printf("\n===== \n");
6         printf("\nTabuada do %d\n", i);
7
8         for (int j = 0; j <= 10; j++) {
9             printf("%d x %d = %d\n", i, j, i * j);
10        }
11    }
12 }
```

- Loop For Quadrado asteriscos

```

2
3  int main()
4  {
5      int n;
6
7      printf("Digite um numero para p tamanho do quadrado: ");
8      scanf("%d", &n);
9
10     for (int i = 0; i < n; i++) //escrevendo nossas linhas
11     {
12         for (int j = 0; j < n; j++) //escrevendo nossas colunas
13         {
14             printf("* ");
15         }
16         printf("\n");
17     }
18     return 0;
19 }

```

```

5
6  printf("Digite um numero para o tamanho do quadrado:
7  scanf("%d", &n);
8
9  → for (int i = 0; i < 3; i++) {
10     for (int j = 0; j < n; j++) {
11         printf("* ");
12     }
13     → printf("\n");
14 }
15
16 → return 0; ✓
17 }

```

0, 1, 2

0	*	*	*
1	*	*	*
2	*	*	*

\n

\n

- While/ Do..While

```

1 while (condição) {
2     // bloco de código
3 }

```

```
do {
    // bloco de código
} while (condição);
```

usado para que o DO seja executado pelo menos um vez (mesmo que o while seja false e não execute, o DO sempre é executado)

- Os tipos de Loop

Tipo de Loop	Quando Usar
for	Quando você sabe o número de repetições. Ex: contar de 1 até 10.
while	Quando não sabe quantas vezes vai repetir, e pode ser zero vezes. Ex: esperar o usuário digitar "sair".
do...while	Quando precisa rodar ao menos uma vez, independente da condição. Ex: pedir senha até acertar.

Funções

```
// tipo_retorno nome_funcao (parametros){
    //bloco de código
return valor; \\ se houver retorno
}
```



```

3_FUNCOES_BIBLIOTECAS > funcoes.c > ...
1  #include <stdio.h>
2  // FUNÇÃO DECLARADA
3  int somar(int numero1, int numero2)
4  { // O VALOR QUE RETORNAR SERA COLOCADO NO soma(), dentro do main
5      return numero1 + numero2;
6  }
7
8  int main()
9  {
10
11      int resultado = somar(3, 2);
12      printf("Resultado: %d", resultado);
13      return 0;
14  }

```

- **Void**

A função Void, é usada por não ter return, não te retorna nada.

```

1 void mostrarMensagem() {
2     printf("Volte sempre!\n");
3 }

```

Bibliotecas

- **Biblioteca Math**

```

biblioteca_math.c U X
3_FUNCOES_BIBLIOTECAS > bibliotecas > biblioteca_math.c > main()
1  /* <math.h> - funções matematicas
2
3  - sqrt() - raiz quadrada ;
4  - pow() - potencia ;
5  - abs((inteiro)) / fabs((float)) - valor absoluto [retorna valor, sem se importa com sinal de operação ];
6  - sin(), cos(), tan() - trigonometria ;
7
8  */
9

```

```

#include <stdio.h>
#include <math.h>

int main()
{
    int base = 2, expoente = 3;

    printf("2 elevado a 3 : %.2f\n", pow(base, expoente));
    printf("Raiz quadrada de 25 : %.2f\n", sqrt(25));

    return 0;
}

```

- **Biblioteca Stdlib**

```

C string.c U ● C stdlib.c U X
3_FUNCOES_BIBLIOTECAS > bibliotecas > C stdlib.c > main()
1  /* <stdlib.h> - utilidade gerais
2
3  - rand() - número aleatório (gera apenas uma vez e salva ele)
4  - srand() - inicializa o gerador com uma semente (gera toda vez que roda o sistema )
5  - atoi(), atof() - converte string para int e float
6  - exit() - encerra o programa
7

```

```

✓ #include <stdio.h>
#include <stdlib.h>
#include <time.h>
//RAND
✓ int main()
{
    // sempre retona o mesmo número quando compila
    int numero = rand() % 100; // entre 0 e 99

    printf("Numero aleatorio : %d\n", numero);

    return 0;
}

```

```
// SRAND
int main()
{
    // retorna número diferentes toda vez quando compila, pois roda
    // em cima do tempo (minutos, segundos), então sempre vai ser alterado;
    srand(time(NULL));
    int numero = rand() % 100; // entre 0 e 99

    printf("Numero aleatorio : %d\n", numero);

    return 0;
}
```

```
// ATOI
int main()
{
    char texto[] = "123";
    int valor = atoi(texto);

    printf("Convertido: %d\n", valor);

    return 0;
}
```

- **Biblioteca string**

```
3_FUNCOES_BIBLIOTECAS > bibliotecas > string.c > main()
1  /* <string.h> - Manipulação de strings
2     PRINCIPAIS FUNÇÕES
3
4  - strlen() - tamanho da string
5  - strcpy() - copia uma string
6  - strcmp() - compara strings
7  - strcat() - concatena strings
8
```

```

#include <stdio.h>
#include <string.h>

int main()
{
    char nome[] = "Joao";
    char sobrenome[] = " Silva";

    strcat(nome, sobrenome); // concatena
    printf("Nome completo: %s\n", nome);

    printf("Tamanho: %d\n", strlen(nome)); // tamanho da string

    if (strcmp(nome, "Joao Silva") == 0) // compara se a string é igual (== 0 , se é TRUE)
    {
        printf("Nome correto!\n");
    }
    else
    {
        printf("Nome incorreto!\n");
    }

    return 0;
}

```

- **Biblioteca Locale**

```

C locale.c U X
3_FUNCOES_BIBLIOTECAS > bibliotecas > C locale.c > main()
1  #include <stdio.h>
2  // chcp 65001 - ativa o UTF-8
3  #include <locale.h> // importar esta biblioteca
4
5  int main()
6  {
7      setlocale(LC_ALL, ""); // habilitar acentos no windows
8      printf("Coração!\n");
9      return 0;

```

Vetores

```
ESTRUTURA_DADOS_BASICAS > vetores.c > main()
1  #include <stdio.h>
2  // LEMBRE-SE, O PRIMEIRO INDICES É O 0 E O ULTIMO É N-1
3  int main()
4  {
5      int notas[5]; // Vetor para armazenar 5 notas
6
7      //-> INT: tipo dos valores (números inteiros);
8      //-> NOTAS: nome do vetor;
9      //-> [5]: número total de elementos que cabem no vetor;
10 }
11 //=====//
```

Mostrando os valores do array, utilizando o FOR, para não ter que fazer vários printf()

```
13 int main()
14 {
15     int notas[5] = {10, 30, 5, 60, 55}; // declaro o tamanho e passei os valores macetados na mão
16
17     for (int i = 0; i < 5; i++)
18     {
19         printf("%d\n", notas[i]); // [i], de índice
20     }
21
22     return 0;
23 }
```

Matrizes

Para passar os valores de uma matriz, sempre passamos os valores das linhas primeiro e depois das colunas. Sempre lembrando que todas começam da posição ZERO .

Exemplo: INT numero [7] [10] (sete linhas e dez colunas, sendo iniciadas do zero)

- **Matriz 3X2 com declaração de valores**

Eu fiz a declaração do tamanho da matriz, e passei os valores delas, seguindo conforme: linhas e colunas...

```
2  int main () {
3
4      int notas [3][2] = {
5          {1 , 2},
6          {10, 6},
7          {7 , 4}
8      };
9
```

Linhas	Colunas
1	2
10	6
7	4

```
int notas[3][2] = {
    {1, 2}, // linha 0 , coluna 0/1
    {10, 6}, // linha 1 , coluna 0/1
    {7, 4} // linha 2 , coluna 0/1
};

printf("%d", notas[2][1]); // aqui estou passando para retornar o valor da matriz...
                           // aqui por exemplo retorna numero 4 (linha 2 coluna 1)

return 0;
```

- **Matriz 3X2 usando FOR para fazer declaração de valores**

```
{
    int notas[3][2]; // agora não terei os valores passados, farei um loop para capturar os valores desejados

    for (int i = 0; i < 3; i++) // linhas
    {
        for (int j = 0; j < 2; j++) // colunas
        {
            printf("Digite a nota do aluno %d, prova %d: ", i + 1, j + 1);
            scanf("%d", &notas[i][j]);
        }
    }
    printf("\n===== NOTAS =====\n");
    for (int i = 0; i < 3; i++) // linhas
    {
        printf("Aluno %d: ", i + 1);
        for (int j = 0; j < 2; j++) // colunas
        {
            printf("%d ", notas[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

```
Digite a nota do aluno 1, prova 1: 8
Digite a nota do aluno 1, prova 2: 5
Digite a nota do aluno 2, prova 1: 10
Digite a nota do aluno 2, prova 2: 10
Digite a nota do aluno 3, prova 1: 3
Digite a nota do aluno 3, prova 2: 2

===== NOTAS =====
Aluno 1: 8 5
Aluno 2: 10 10
Aluno 3: 3 2
```

Variáveis Globais x Locais

- **Global:** Do lado de fora de uma loop, onde pode acessar as variáveis de qualquer lugar, pode acessar em qualquer momento de qualquer lugar;
- **Local:** Coloca dentro de um escopo, encapsulada de um local, o que aconteceu em um loop FOR, com índice = i, a variável fica lá dentro... o que podemos criar duas vezes estruturar com i e j, e não interfere em nada!

Ponteiros e Alocação Dinâmica de Memória

- **Introdução Ponteiros:** Ao invés de ficar levando o “pacote” (valor da variável) para vários lugares diferentes, o que fica mais barato, eu utilizo o ponteiro que aponta para ele, e mostro o endereço, ao invés de pegar ele. Ou seja, o ponteiro é o **ENDEREÇO** e não o **VALOR**;
- **Alocação Dinâmica de Memória: como se estivesse “alugando memória” no tempo de execução e temos: FREE : devolve;**

MALLOC: aluga;

```
1  #include <stdio.h>
2  #include <stdlib.h> // para usar o malloc
3
4  int main()
5  {
6      int tamanho;
7      int *vetor;
8
9      printf("Digite o tamanho do vetor: ");
10     scanf("%d", &tamanho);
11
12     vetor = (int *)malloc(tamanho * sizeof(int));
13
14     if (vetor == NULL)
15     {
16         printf("Erro: memoria insuficiente.");
17         return 1;
18     }
19
20     for (int i = 0; i < tamanho; i++)
21     {
22         vetor[i] = i * 10;
23     }
24
25     printf("Vetor dinamico populado: \n");
26
27     for (int i = 0; i < tamanho; i++)
28     {
29         printf("%d\n", vetor[i]);
30     }
31
32     free(vetor);
33
34     return 0;
```

CALLOC: aluga e limpa;(zera tudo e depois aloca)

```
int main()
{
    int tamanho;
    int *vetor;

    printf("Digite o tamanho do vetor: ");
    scanf("%d", &tamanho);

    vetor = (int *)calloc(tamanho, sizeof(int));

    if (vetor == NULL)
    {
        printf("Erro: memoria insuficiente.");
        return 1;
    }

    for (int i = 0; i < tamanho; i++)
    {
        vetor[i] = i * 10;
    }

    printf("Vetor dinamico populado: \n");

    for (int i = 0; i < tamanho; i++)
    {
        printf("%d\n", vetor[i]);
    }

    free(vetor);

    return 0;
}
```


- **Segmentation fault** : aponta para um endereço que não possui valor nenhum

```
#include <stdio.h>

int main()
{
    int numero = 42;
    int *n = &numero; // ponteiro
    // este caso ele esta apontando (&) para o endereço de memoria da variavel numero

    printf("Antes: %d\n", numero); // aqui ele trazendo o endereço que foi passando pelo ponteiro da
    variavel nuemro = 42

    *n = 10; // ele este mudando o endereço da varivel numero, alterando o valor de 42 para 10

    printf("Depois: %d\n", numero);

    return 0;
}
```

Struct(estrutura)

É um tipo de dado composto que agrupa diferentes variáveis (campos) relacionadas sob um único nome, permitindo tratar esses dados como uma única unidade, como os dados de um "Aluno" (nome, idade, média) ou uma "Data" (dia, mês, ano) em uma única variável.

- **Sintaxe**, sempre utilizado por fora da main

```
1 struct Pessoa // pessoa é um escopo -> capsula
2 {
3     char nome[50];
4     int idade;
5     float altura;
6 };
7
8 int main()
9 {
10     struct Pessoa p1; // p1 é a variavel da estrutura Pessoa que foi criada
11
12     // para utilizar, digita p1 e ponto e o proprio teclado, vai dar as opções que tem dentro
13     dostructPessoa
14     printf("Digite o nome: ");
15     scanf("%s", &p1.nome);
16
17     printf("Digite a idade: ");
18     scanf("%d", &p1.idade);
19
20     printf("Digite a altura: ");
21     scanf("%f", &p1.altura);
22
23     printf("Nome: %s\n", p1.nome);
24     printf("Idade: %d\n", p1.idade);
25     printf("Altura: %.2f\n", p1.altura);
26
27     return 0;
28 }
29
30 }
```

- **Ponteiros**

```
3 struct Pessoa // pessoa é um escopo -> capsula
4 {
5     char nome[50];
6     int idade;
7     float altura;
8 };
9
10 int main()
11 {
12
13     struct Pessoa p1;
14     struct Pessoa *ponteiro = &p1; // criação do ponteiro e fazendo o endereço de onde vai buscar
15
16     printf("Digite o nome: ");
17     scanf("%s", ponteiro->nome); // como ir buscar o endereço pelo ponteiro (ponteiro->)
18
19     printf("Digite a idade: ");
20     scanf("%d", &ponteiro->idade);
21
22     printf("Digite a altura: ");
23     scanf("%f", &ponteiro->altura);
24
25     printf("Nome: %s\n", ponteiro->nome);
26     printf("Idade: %d\n", ponteiro->idade);
27     printf("Altura: %.2f\n", ponteiro->altura);
28
29     return 0;
30 }
```

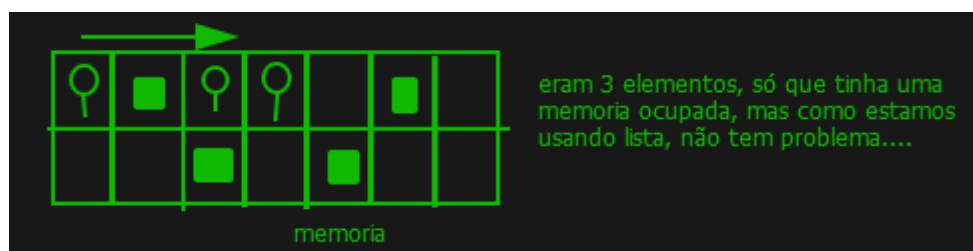
Union

É um tipo de dado definido pelo usuário que permite armazenar diferentes tipos de dados no mesmo local de memória, economizando espaço, mas apenas um membro pode ser usado por vez, pois todos compartilham a mesma área, cujo tamanho é o do maior membro, sendo útil para quando você precisa usar diferentes tipos de dados que não são necessários simultaneamente.

Listas

- **Encadeadas**, estrutura de dados dinâmica onde elementos (nós) são ligados por ponteiros, não precisando de memória contínua, ao contrário de arrays, permitindo inserções e remoções flexíveis ao apenas ajustar referências.

(Eles sabem qual é o próximo elemento, por isso ele busca mesmo tendo o espaço da memória entre eles, o que no array, não funcionaria, pois deveria ser uma linha contínua)



```
lista_encadeada-1.c U X
6 ESTRUTURA_AVANÇADAS > lista_encadeada-1.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct No
5  {
6      int valor;
7      struct No *proximo;
8  };
9
10 int main()
11 {
12
13     struct No *primeiro = (struct No *)malloc(sizeof(struct No));
14
15     primeiro->valor = 42;
16     primeiro->proximo = NULL;
17
18     printf("Valor do no: %d\n", primeiro->valor);
19     printf("Valor do proximo - no: %s\n", primeiro->proximo);
20
21     return 0;
22 }
```

- Loop

```

4
5 struct No
6 {
7     int valor;
8     struct No *proximo;
9 };
10
11 int main()
12 {
13
14     int tamanhoLista;
15
16     printf("Digite o tamanho da lista: ");
17     scanf("%d", &tamanhoLista);
18
19     if (tamanhoLista <= 0)
20     {
21         printf("valor invalido");
22         return 1;
23     }
24
25     struct No *primeiro = NULL;
26     struct No *ultimo = NULL;
27
28     for (int i = 0; i < tamanhoLista; i++)
29     {
30         struct No *novo = (struct No *)malloc(sizeof(struct No));
31
32         printf("Digite o valor do No %d: ", i);
33         scanf("%d", &novo->valor);
34
35         novo->proximo = NULL;
36
37         if (primeiro == NULL)
38         {
39             primeiro = novo;
40             ultimo = novo;
41         }
42         else
43         {
44             ultimo->proximo = novo;
45             ultimo = novo;
46         }
47     }
48     printf("Valores da lista: ");
49
50     struct No *temp = primeiro;
51     while (temp != NULL)
52     {
53         printf("%d ", temp->valor);
54         temp = temp->proximo;
55     }
56
57     // limpar memoria
58
59     temp = primeiro;
60     while (temp != NULL)
61     {
62         struct No *proximo = temp->proximo;
63         free(temp);
64         temp = proximo;
65     }
66

```

Pilha

Pense em uma pilha de pratos.

- LIFO: conceito o qual o último ao ser colocado será o primeiro a ser usado;

FUNÇÕES UTILIZADAS NO CONCEITO PILHA

- Push: empurrar (colocar um prato no topo da pilha)
- Pop: retirar (sempre retirar o último da pilha, do topo);
- Top: pegar o último, e consumir o'que ele está fazendo.

PUSH:

```
1  #include <stdio.h>
2
3  int tamanho_maximo = 10;
4  int pilha[10];
5  int topo = -1;
6
7  void push(int valor)
8  {
9      if (topo == tamanho_maximo - 1)
10     {
11         printf("Erro: pilha cheia\n");
12         return;
13     }
14
15     topo++;
16     pilha[topo] = valor;
17 }
18
19 void imprimirPilha()
20 {
21     if (topo == -1)
22     {
23         printf("Pilha vazia\n");
24         return;
25     }
26     printf("Elemento da Pilha: \n");
27     for (int i = topo; i >= 0; i--)
28     {
29         printf("%d\n", pilha[i]);
30     }
31 }
32
33 int main()
34 {
35
36     push(10);
37     push(67);
38
39     imprimirPilha();
40
41     return 0;
42 }
```

TOP:

```
18
19 int top()
20 {
21     if (topo == -1)
22     {
23         printf("Pilha vazia\n");
24         return -1;
25     }
26     return pilha[topo];
27 }
28
29 void imprimirPilha()
30 {
31     if (topo == -1)
32     {
33         printf("Pilha vazia\n");
34         return;
35     }
36
37     printf("Elemento da Pilha: \n");
38     for (int i = topo; i >= 0; i--)
39     {
40         printf("%d\n", pilha[i]);
41     }
42 }
43
44 int main()
45 {
46     push(10);
47     push(67);
48     push(99);
49
50     imprimirPilha();
51
52     printf("Topo atual %d\n", top());
53
54     return 0;
55 }
56
```

POP:

```
28
29 int pop()
30 {
31     if (topo == -1)
32     {
33         printf("Pilha vazia\n");
34         return -1;
35     }
36     topo--;
37 }
38
39 void imprimirPilha()
40 {
41     if (topo == -1)
42     {
43         printf("Pilha vazia\n");
44         return;
45     }
46
47     printf("Elemento da Pilha: \n");
48     for (int i = topo; i >= 0; i--)
49     {
50         printf("%d\n", pilha[i]);
51     }
52 }
53
54 int main()
55 {
56
57     push(10);
58     push(67);
59     push(99);
60
61     imprimirPilha();
62
63     printf("Topo atual %d\n", topo());
64
65     pop();
66     printf("Topo atual %d\n", topo());
67
68     return 0;
69 }
```

Fila

Primeiro a chegar é o primeiro a sair e assim sucessivamente;

FUNÇÕES UTILIZADAS NO CONCEITO Fila

- Enqueue(), entra no fim (é adicionado no final da fila);
- Dequeue(), remove do início (remove o primeiro elemento da fila)

Enqueue() :

```
2
3  int fila[10];
4  int inicio = 0;
5  int fim = 0;
6  int tamanho_maximo = 10;
7  void enqueue(int valor)
8  {
9      if (fim == tamanho_maximo)
10     {
11         printf("Fila cheia!\n");
12         return;
13     }
14     fila[fim] = valor;
15     fim++;
16 }
17
18 int main()
19 {
20
21     enqueue(1);
22     enqueue(28);
23     enqueue(7);
24     enqueue(100);
25
26     printf("Primeiro elemento da fila: %d", fila[0]);
27     printf("Ultimo elemento da fila: %d", fim);
28 }
```

Dequeue() :

```
3  void dequeue()
4  {
5      if (inicio == fim)
6      {
7          printf("fila vazia!\n");
8          return;
9      }
10
11     for (int i = 0; i < fim - 1; i++)
12     {
13         fila[i] = fila[i + 1];
14     }
15     fim--;
16 }
17
18 int main()
19 {
20
21     enqueue(1);
22     enqueue(28);
23     enqueue(7);
24     enqueue(100);
25
26     printf("Primeiro elemento da fila: %d \n", fila[0]);
27     printf("Ultimo elemento da fila: %d\n", fim);
28
29     dequeue();
30     printf("Primeiro elemento da fila: %d\n", fila[0]);
31     printf("Ultimo elemento da fila: %d\n", fim);
32 }
```


Ordenação

- **Bubble Sort(por bolha):** Ordenação por pares, onde cada vez que eu iterar a minha lista inteira, a bolinha que tiver maior número sobe primeiro e assim sucessivamente, até ordenar tudo

Por exemplo: em um array de [5], pega a posição 0 e 1 e compare qual é o maior valor, o maior valor vai para primeira posição, resolvendo assim a primeira comparação. A próxima comparação é feita pela posição 1 e 2 e assim vai até chegar na última posição, garantindo o último número mais da bolha. Caso não tenha, ficado tudo ordenado ele passa ordenado novamente e fazendo comparações

- **Insert Sort(por inserção):** em vez de iniciar pela índice 0, inicia pelo índice 1, fazendo a comparação se é maior ou não, se for tem troca, caso não haja permanece normal, caso haja um troca, ele já compara, com o número que passou, ele resolve de uma vez as ordem que já passou ,em vez de passar em todos os índices e voltar.. ele já valida tudo, antes de chegar no último.
- **Select Sort(por seleção):** ele passa por todas as posições procurando a menor dela , para por na primeira posição e faz assim em todas
- **Quick Sort(por seleção):** precisa achar o Pivô(geralmente pegamos a quantidade de elementos, dividimos por dois e pegamos a posição do meio), com isso, números maiores que o pivô vai para direita dele e os menor vai para esquerda. Quando acabar os núemro dessa forma e ainda estiveres alguns desordenados, pega só aqueles que estão fora de ordem

Recursão

Ato de criar uma função que irá chamar a si própria(parece um loop, mas é o ato de chamar uma função)

Caso Base -> Fim da recursão

Manipulação de arquivos

Modo	O que faz
"r"	Abre para **Leitura** (erro se o arquivo não existir)
"w"	Abre para **escrita** (cria o arquivo ou apaga o conteúdo anterior)
"a"	Abre para **acrescentar** (escreve no final, sem apagar o conteúdo)
"r+"	Leitura e escrita (arquivo deve existir)
"w+"	Escrita e leitura (apaga o conteúdo se o arquivo já existir)
"a+"	Leitura e acrescentar (mantém conteúdo, adiciona ao final)

